Classical Fourth-Order Runge–Kutta

Four-stage explicit integration

1 Butcher tableau and stages

The classical method has tableau

$$\begin{array}{c|cccc} 0 & 0 & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array}$$

and, for $\dot{z} = f(t, z)$,

$$k_1 = f(t_n, z_n), \tag{1}$$
$$k_2 = f(t_n + h/2, z_n + hk_1/2), \tag{2}$$
$$k_3 = f(t_n + h/2, z_n + hk_2/2), \tag{3}$$
$$k_4 = f(t_n + h, z_n + hk_3), \tag{4}$$
$$z_{n+1} = z_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \tag{5}$$

The non-autonomous stage times are part of the model.

2 Order and stability

The tableau satisfies the Runge–Kutta order conditions through order four; the local error is $\mathcal{O}(h^5)$ and the global error is $\mathcal{O}(h^4)$ for sufficiently smooth problems. On $y' = \lambda y$, its stability polynomial is

$$R(\xi) = 1 + \xi + \frac{\xi^2}{2} + \frac{\xi^3}{6} + \frac{\xi^4}{24}.$$

The finite stability region does not make RK4 an A-stable method.

3 Geometric properties

Classical RK4 is not symmetric and is not symplectic for a general Hamiltonian system. Its high local accuracy can nevertheless make short-time energy and area errors small. The distinction is structural: symplecticity is the exact identity

$$(D\Phi_h)^T \Omega D\Phi_h = \Omega,$$

not merely a small defect at one step size. The implementation exposes each complete step map to observers so local and accumulated defects can be measured.

4 Energy-tracking configuration

An optional `EnergyObserver` is supplied to `simulate` through `observers`. Execution binds the Hamiltonian functions and the observer uses the retained RK4 stages to integrate the passive equation

$$\dot{k} = -\partial_t H(t, z), \qquad K(z, t, k) = H(t, z) + k.$$

The same RK4 nodes and weights are used for z and the normalized momentum. The physical trajectory is returned normally; k and the maximum sampled drift of K are diagnostics. This extension monitors generalized energy but does not turn RK4 into a symplectic or energy-preserving method.

5 Implementation correspondence

The method is implemented in `src/methods/classical/rk4.py` and consumes the injected vector field $f(t, z)$. Main-grid steps are independent of the output schedule; shadow steps provide off-grid samples without changing the accepted trajectory or emitting observations. See the sibling runtime document for the exact result and diagnostic contract.

6 Physical accepted states and passive energy tracking

For N particles with d physical components, every accepted snapshot contains $Z \in \mathbb{R}^{dN}$ and one scalar time t. An optional energy observer owns separate physical momentum $\kappa \in \mathbb{R}^N$. Thus GC accepted vectors have $2N$ components and FC vectors have $4N$, whether energy is tracked or not. A doubled method prepares temporary $(u, v) \in \mathbb{R}^{2dN}$ copies, allows them to separate, and projects to one physical result before acceptance. The representation stores only the physical size and never owns a clock, momentum, physical coordinate declaration, dynamics or solver.

The normalized passive balance is

$$\dot{\kappa} = -\partial_t H(t, Z), \qquad \kappa(t_0) = 0, \qquad \Delta K(t) = H(t, Z(t)) + \kappa(t) - H(t_0, Z_0).$$

GC split-map contributions are summed in temporary work and divided by two to obtain physical momentum. Tracking leaves physical stages, nonlinear stopping scales and adaptive acceptance unchanged. An autonomous Hamiltonian supplies an explicit zero momentum derivative.

Hairer's constraint is solely $u - v = 0$: the reduced multiplier has $2N$ entries and ABBA's simultaneous spatial solve has $6N$ unknowns. Neither t nor κ is projected. The former eight-component projection is retired. Dynamics declare component order and position/velocity roles through immutable structural data; initial conditions and results retain those data independently. Execution injects $f(t, Z)$ and optional physical derivatives into a fresh method run. Methods own physical stages and projection equations, without retaining the dynamics object. Nonlinear tolerances, iteration limits, differentiation and iteration callbacks belong to the configured algebraic solver. Generic doubled copy operations live in `src/formulations/state.py`; classical, HBVM and adaptive methods need no state formulation.

Methods optionally return retained stage quadrature or a dense interpolant. The energy observer integrates momentum independently, using Hamiltonian functions supplied by execution. Controllers keep only physical accepted states; compiled JAX loops carry the observer accumulator separately. Observers receive physical states. Energy histories in `States.diagnostics` retain their existing labels; `extended_time` has shape (N, n_{samples}) and is broadcast from scalar sample times. Fixed shadow samples leave accepted states and counters unchanged. Adaptive quadrature uses accepted dense output and physical-only error control.

The passive momentum uses the same four stage states and weights as the physical RK4 update.

7 CPU and JAX execution of the same map

The execution configuration selects either the default NumPy/SciPy CPU driver or a JAX driver on one CPU or GPU device. Both consume the same RK4 stage and passive-quadrature formulas.

The potential uses the same fitted spline knots and coefficients in both implementations. JAX requires explicit double precision; different floating-point operation ordering can produce round-off differences, so equivalence does not mean bitwise identity.

For independent particles, stage operations are batched over the particle index. The four stages retain their dependencies, and successive time steps remain sequential inside a compiled loop. The accepted state and requested sample buffer remain on the chosen device until the completed result is transferred to the standard immutable NumPy solution. Independent shortened maps still provide off-grid samples without altering accepted main states. The passive momentum and sampled Hamiltonian are evaluated on that device.

The compiled route currently covers the built-in guiding-centre and full-cyclotron equations. Custom dynamics, method subclasses and Python step callbacks or progress reporting require the CPU route. Device choice does not change the order, stability region or geometric properties derived above. First-call compilation must be distinguished from warmed integration timings, which include transfers and output construction in the test study.